

TTB Label Verification App

Architecture & Build-Prompt Playbook

A step-by-step guide to building the app with OpenAI Codex in Visual Studio Code — written for developers new to Codex. Covers setup, the Plan → Review → Execute workflow, the architecture, and the exact prompts to run.

Contents

Part 0 — Getting Started with OpenAI Codex (in VS Code).....	2
What You Need First (prerequisites)	2
Step-by-Step Setup	2
The Three Codex Modes = Plan, Review, Execute	2
How to Run One Phase (do this for each phase in Part 3)	3
Tips & Troubleshooting	3
Part 1 — Architecture	5
System Overview	5
Components & Responsibilities	5
Data Model	5
Comparison Strategy	6
Build Order (dependency-driven).....	6
Part 2 — How to Use the Prompts	7
Standing Rules (set up once, before Phase 0).....	7
Part 3 — Phase-by-Phase Prompt Sequences.....	8
Phase 0 — Scaffold, Secrets, Deploy Skeleton	8
Phase 1 — Data Models + Comparison Engine (pure logic, TDD).....	8
Phase 2 — Vision Service (extraction)	8
Phase 3 — /verify Endpoint (wire extraction + comparison)	9
Phase 4 — Frontend Single-Label Flow	10
Phase 5 — Batch Upload.....	10
Phase 6 — Robustness, Performance, Accessibility	10
Phase 7 — Deploy Verification + README + Submission	10
Notes on Using This Playbook.....	11

Part 0 — Getting Started with OpenAI Codex (in VS Code)

If you have never used OpenAI Codex, work through it once, top to bottom. Codex is an AI coding agent that lives inside Visual Studio Code: you talk to it in a side panel, and it reads your project, writes code, and runs commands for you — with your approval. The rest of this document (Parts 1–3) is what you feed it.

The big idea: you will not type much code yourself. You will paste the prompts from Part 3 into Codex, review what it proposes, and approve it. The Plan → Review → Execute loop keeps Codex on a short leash so it builds the right thing in the right order.

What You Need First (prerequisites)

- **Visual Studio Code** installed (free) — code.visualstudio.com.
- **An OpenAI account on a paid ChatGPT plan** (a license has been provisioned and you have been invited) — Codex access is included.
- **Git and a GitHub account** — Git installed locally; the final deliverable is a repo.
- **Python 3.12 and Node.js 22+** on your machine — these run the app itself (backend and frontend). Codex uses them when it runs your tests.

You do not need to be fluent in any of these — Codex handles most commands. You just need them installed so the commands work.

Step-by-Step Setup

1. **Install the Codex extension.** Open VS Code → the Extensions panel (the squares icon in the left bar) → search for “Codex” (publisher: OpenAI; the listing is “ChatGPT – Codex”) → Install. You can also run this in VS Code’s Quick Open (Ctrl/Cmd-P):

```
ext install openai.chatgpt
```

2. **Open the Codex panel and sign in.** Click the Codex icon in the left activity bar. In the panel, click “Sign in with ChatGPT.” Your browser opens; log in to your OpenAI account and approve. The panel will show you’re connected. (If you prefer an API key, choose that option and paste your key instead.)
3. **Create the project folder and open it in VS Code.** Make an empty folder for the app, then in VS Code use File → Open Folder and select it. Open a terminal (Terminal → New Terminal) and initialize git:

```
git init
```

4. **Add the project rules file (AGENTS.md).** Create a new file named AGENTS.md in the root of the folder and paste the “Standing Rules” block from Part 2 of this document into it. Codex automatically reads AGENTS.md at the start of every conversation, so these rules apply to the whole build — you only paste them once, here. (You will paste the per-phase prompts into the chat as you go; the rules stay in AGENTS.md.)
5. **You’re ready.** From here you work phase by phase (Part 3), running the Plan → Review → Execute loop described next.

The Three Codex Modes = Plan, Review, Execute

Codex has a mode selector at the top of its panel with three approval levels. They map directly onto our loop — this is the single most important thing to understand:

Codex mode	What Codex can do	Use it for
Chat (read-only)	Read your files and answer. Cannot edit files or run commands.	PLAN and REVIEW — it can study the project and propose/critique a plan, but can't change anything yet.
Agent (default)	Read, edit files, and run commands (e.g. tests) inside your project folder. Asks approval to go outside it.	EXECUTE — implementing the approved plan and running the tests.
Agent — Full Access	Adds internet access (fetch docs, reach external services).	Only when a phase truly needs the network (e.g. installing packages or deploying). Switch back afterward.

Why this matters: running PLAN/REVIEW in read-only mode guarantees Codex cannot start writing code before you have approved the plan. That gate is the whole point of the method.

How to Run One Phase (do this for each phase in Part 3)

1. **PLAN.** Set the mode selector to Chat (read-only). Copy the phase's PLAN prompt from Part 3, paste it into the Codex panel, and send. Read the plan it returns. If the scope looks wrong or too big, reply in plain English to correct it before moving on.
2. **REVIEW.** Stay in Chat (read-only). Paste the phase's REVIEW prompt. Codex checks its own plan against the requirements and edge cases. For a tougher, independent check, start a new Codex conversation first (the "+" / new-chat button) so it reviews with fresh context.
3. **EXECUTE.** Switch the mode selector to Agent. Paste the phase's EXECUTE prompt. Codex will now propose file edits and run commands. Review each diff it shows and click Approve (or reject and tell it what to change). It will run the tests itself.
4. **Verify the exit check.** Every EXECUTE prompt ends with an "Exit check." Confirm it's true — run the app or tests yourself if you want to be sure. Do not start the next phase until it passes.
5. **Commit your work.** In the terminal, save a checkpoint so you can always roll back:

```
git add -A && git commit -m "Phase 1 complete: comparison engine + tests"
```
6. **Move to the next phase** and repeat from step 1.

Tips & Troubleshooting

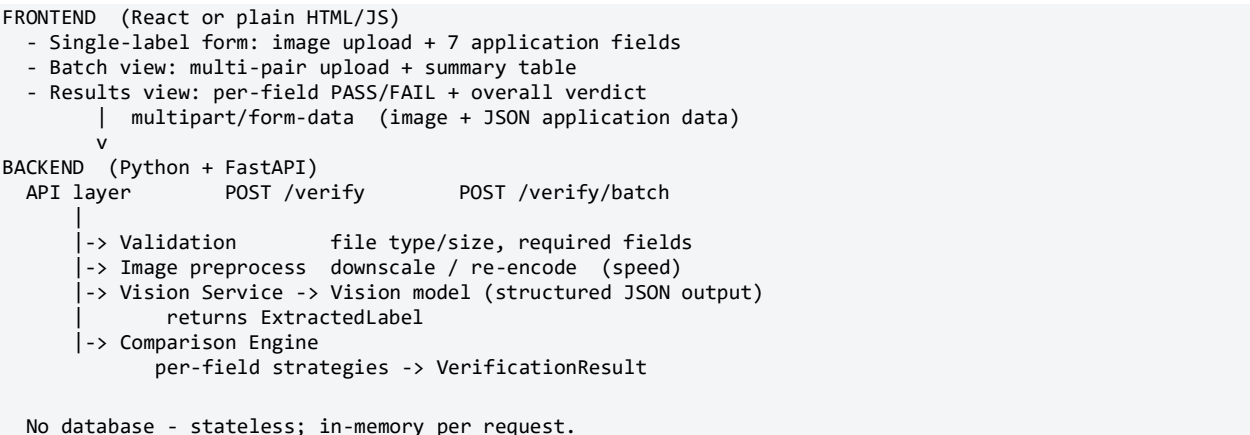
- **Never paste real API keys into the chat.** Keep them in a local .env file (Phase 0 sets this up) and make sure .gitignore excludes it. A key committed to GitHub is an automatic disqualifier.
- **If Codex changes more than the current phase,** reject the diff, or undo with git (git restore . or git checkout .), and re-send the prompt adding "only do Phase N; do not touch anything else."
- **Read the diffs before approving.** Codex shows exactly what it will change. Approving blindly is how bugs and scope creep slip in.

- **Use a fresh conversation for REVIEW** when you want a genuinely independent second look — a brand-new chat re-reads the code without being anchored to its earlier plan.
- **If a command fails**, paste the error back into the Codex panel and ask it to diagnose and fix — that is exactly what it's good at.
- **Usage counts against your ChatGPT plan limits.** Long EXECUTE runs use more; the Plan/Review steps are cheap. Commit often so a hiccup never costs you a whole phase.

Part 1 — Architecture

System Overview

A thin web frontend collects an image plus structured application data and posts it to a stateless FastAPI backend. The backend validates input, preprocesses the image, calls a vision model for structured extraction, then runs the comparison engine field-by-field and returns per-field PASS/FAIL plus an overall verdict. No database — each request is self-contained.



Components & Responsibilities

Component	Responsibility	Why isolated
Comparison Engine	Field-by-field match logic (fuzzy / normalized / exact)	Pure functions, zero I/O → fully unit-testable without the model. Build first.
Vision Service	Image → structured ExtractedLabel via a vision model	The only slow, costly, nondeterministic part. Isolate so it can be mocked in tests.
API layer	HTTP, validation, orchestration, error shaping	Thin; delegates to the two services above.
Image preprocess	Downscale / re-encode before the model call	Directly serves the 5-second budget and cost control.
Frontend	Upload, form, results, batch summary	Built against a stable API contract.
Config / secrets	API keys via environment variables only	Hard requirement; committing a key is an auto-disqualifier.

Data Model

Pydantic models are the contract everything else builds on:

ApplicationData	brand_name, class_type, abv, net_contents, producer, country_of_origin, government_warning
ExtractedLabel	same 7 fields + raw_text + extraction_confidence (fields nullable - image may omit/obscure them)
FieldResult	field, match_type, expected, found, status: PASS FAIL
VerificationResult	results: list[FieldResult], overall_verdict: APPROVED NEEDS_REVIEW, latency_ms
BatchResult	items: list[VerificationResult], summary: {passed, needs_review, total}

Verdict rule (make it explicit): any field FAIL ⇒ NEEDS_REVIEW; all PASS ⇒ APPROVED.

Comparison Strategy

The heart of correctness. Each field gets a deliberate strategy:

Field	Strategy	Implementation note
Brand Name, Class/Type, Producer	Fuzzy	Normalize case/punctuation/whitespace, then a fuzzy ratio ≥ a stated threshold (e.g. 90).
Country of Origin	Normalize + synonym	Map aliases (USA / U.S.A. / United States) before comparing — pure fuzzy fails here.
Alcohol Content (ABV)	Numeric normalize	Extract the numeric value; 45% = 45.0% = “45% Alc./Vol.”; tolerance ±0.1.
Net Contents	Unit normalize	Convert to a canonical unit (e.g. mL) before compare; 750 mL == 750ml.
Government Warning	EXACT	Whitespace-collapse only; case-sensitive; no fuzzy. Compare to the canonical statement.

Critical tension to design for: the warning is exact-match, but images are imperfect. A single OCR misread should not silently hard-FAIL. Always return the extracted warning text alongside the FAIL so an agent can eyeball-override.

Build Order (dependency-driven)

This is also the phase order used for the prompts in Part 3:

0	Scaffold + secrets + deploy skeleton	(deploy EARLY)
1	Data models + Comparison Engine	(pure, TDD, no AI)
2	Vision Service	(prompt + structured output + preprocess)
3	/verify endpoint	(wire 1 + 2)
4	Frontend single-label flow	
5	Batch (backend concurrency + frontend summary)	
6	Robustness + performance + accessibility	
7	Deploy verify end-to-end + README	

Part 2 — How to Use the Prompts

Each phase runs the same three-beat loop. Treat the prompts as three separate messages to your AI coding agent, with a human gate between each:

- **PLAN** — the agent reads context and proposes an approach. No code yet. You read it, sanity-check against the requirements, and adjust scope.
- **REVIEW** — the agent (or a fresh agent session, for independent eyes) critiques its own plan against the requirements and edge cases, and finalizes it.
- **EXECUTE** — the agent implements only the approved plan, writes tests, and self-verifies. Do not move on until the phase exit check is green.

The gate between Review and Execute is the point: it stops the agent jumping straight from a plan to a large, untested code drop — which is where scope creep and missed edge cases enter.

Standing Rules (set up once, before Phase 0)

Save this block as AGENTS.md in your repository root (see Part 0, Step 4). Codex reads AGENTS.md automatically at the start of every conversation, so these rules apply to the whole build and you only paste them once:

PLAN

You are helping build the TTB Label Verification proof-of-concept. Standing rules for the whole project: 1. Stack: Python 3.12 + FastAPI backend, React (or plain HTML/JS) frontend, a vision model for extraction, no database (stateless / in-memory). Deploy target: a free-tier host (e.g. Railway, or Vercel + Render). 2. Hard requirements that override convenience: single-label result in UNDER 5 SECONDS; UI usable by a non-technical 70+ user with no instructions; BATCH UPLOAD is required, not optional; the government warning is an EXACT, case-sensitive match while all other fields are fuzzy/normalized; API keys live in ENVIRONMENT VARIABLES ONLY — never hardcoded, never committed. 3. Working cadence: when I say PLAN, propose an approach and list files/risks but write NO code. When I say REVIEW, critique that plan against the requirements and edge cases and finalize it. When I say EXECUTE, implement exactly the approved plan with tests, then tell me how to verify it. Keep scope to the current phase only. 4. Prefer correctness and clean structure over ambition. Confirm you understand these rules.

(The PLAN tag on the block above is only styling — this text is your AGENTS.md file, not a phase prompt. The actual per-phase prompts are in Part 3.)

Part 3 — Phase-by-Phase Prompt Sequences

Phase 0 — Scaffold, Secrets, Deploy Skeleton

PLAN

PLAN Phase 0. Propose a minimal repo structure for a FastAPI backend + a simple frontend, deployable to a free-tier host. Include: directory layout, dependency manager (uv), an .env.example + .gitignore strategy for secrets, a /health endpoint, and a “hello” frontend that calls it. Goal: a live deployed URL BEFORE any real features. List the files you would create and the deploy steps. No code yet.

REVIEW

REVIEW the Phase 0 plan. Confirm: secrets can never be committed (.gitignore covers .env), the deploy path actually works on the chosen free tier, and CORS is handled between frontend and backend. Flag anything that would bite us at deploy time. Finalize the plan.

EXECUTE

EXECUTE Phase 0. Scaffold the repo, add .env.example, wire /health, and a frontend page that fetches it. Give me the exact commands to run locally and to deploy. Exit check: the app loads at a live URL and the frontend shows the health response.

Phase 1 — Data Models + Comparison Engine (pure logic, TDD)

PLAN

PLAN Phase 1. We are building the comparison engine with NO AI yet – pure functions over typed data, so it is fully unit-testable. Propose: the Pydantic models (ApplicationData, ExtractedLabel, FieldResult, VerificationResult), and one comparison function per field with its strategy – fuzzy (brand/class/producer), country-with-synonyms, ABV numeric-normalize, net-contents unit-normalize, and government-warning exact case-sensitive. State the fuzzy library + threshold and the verdict rule (any FAIL ⇒ NEEDS_REVIEW). List the test cases you will write. No code yet.

REVIEW

REVIEW the Phase 1 plan against these exact scenarios and confirm each is covered by a planned test: case-only brand diff passes; “45%” vs “45% Alc./Vol. (90 Proof)” passes; “750 mL” vs “750ml” passes; “USA” vs “United States” passes; government warning in title case FAILS; warning missing the colon FAILS; correct all-caps warning PASSES; a misread warning returns the extracted text in the result. Add any missing tests. Finalize.

EXECUTE

EXECUTE Phase 1 test-first: write the tests, then the models and comparison functions until tests are green. No network calls. Exit check: all tests pass and government_warning uses strict case-sensitive comparison (show me that test).

Phase 2 — Vision Service (extraction)

PLAN

PLAN Phase 2. Design a VisionService that takes an image and returns an ExtractedLabel. Cover: the vision model choice and how we get STRUCTURED JSON output (not string parsing); the extraction prompt (instruct it to return the 7 fields, leave unknowns null, and copy the government warning verbatim); image preprocessing (downscale / re-encode to protect the 5-second budget); graceful handling of blurry/angled/glare images (return partial data, do not throw); and how we mock this service in tests. No code yet.

REVIEW

REVIEW the Phase 2 plan. Pressure-test: does the prompt force verbatim capture of the warning (critical for exact match)? What happens on a non-label image or a model timeout? Is the structured-output parse defensive against malformed JSON? Confirm the service is mockable so Phase 3 tests do not hit the API. Finalize.

EXECUTE

EXECUTE Phase 2. Implement VisionService with structured output + preprocessing + defensive parsing, plus a fake/mock implementation for tests. Add a small script to run it against one sample image. Exit check: a real sample image returns a populated ExtractedLabel; tests use the mock and pass.

Phase 3 — /verify Endpoint (wire extraction + comparison)

PLAN

PLAN Phase 3. Design POST /verify accepting an image + application data (multipart) and returning a VerificationResult. Cover: request validation (file type/size, required fields), orchestration (preprocess → extract → compare → verdict), latency measurement returned in the response, and error shaping (human-readable messages, never stack traces). List the endpoint tests using the mocked VisionService. No code yet.

REVIEW

REVIEW the Phase 3 plan. Confirm: bad file type and empty submissions return clear 4xx errors (not 500s); the response includes per-field results AND expected-vs-found on failures AND the overall verdict AND latency; the warning's extracted text is surfaced. Check the 5-second budget is measured and logged. Finalize.

EXECUTE

EXECUTE Phase 3. Implement the endpoint and its tests (mocked vision). Exit check: tests green; a curl/HTTP example returns a full VerificationResult; invalid input returns a readable error.

Phase 4 — Frontend Single-Label Flow

PLAN

PLAN Phase 4. Design the single-label UI for a non-technical 70+ user: image picker + the 7 clearly-labeled fields + a big submit, then a results view showing per-field PASS/FAIL with expected-vs-found on failures and a prominent APPROVED / NEEDS REVIEW verdict. Cover loading state, human-readable error display, and large/clear/high-contrast controls. No code yet – describe the layout and the API call.

REVIEW

REVIEW the Phase 4 plan against the “70-year-old, no instructions, under 30 seconds” bar: is the primary action obvious, is jargon gone, are errors plain-English, is the failing-field reason immediately clear? Flag anything that needs hunting. Finalize.

EXECUTE

EXECUTE Phase 4. Build the single-label page wired to /verify on the deployed backend. Exit check: I can upload an image + data on the live URL and see per-field results and a verdict; errors render readably.

Phase 5 — Batch Upload

PLAN

PLAN Phase 5. Design POST /verify/batch for multiple image+data pairs and a frontend batch view with a summary (passed / needs-review / total) plus drill-down to each individual result. Address the 5-second budget at scale: process labels CONCURRENTLY (async), and show a progress indicator when processing takes more than a moment. No code yet.

REVIEW

REVIEW the Phase 5 plan. Confirm: one bad label in a batch does not fail the whole batch (per-item error isolation); concurrency is bounded to respect API rate/cost; the summary counts are correct; each item is still individually viewable. Finalize.

EXECUTE

EXECUTE Phase 5. Implement the batch endpoint (concurrent) + tests, and the frontend summary + drill-down + progress indicator. Exit check: 3+ labels process together on the live URL; summary counts are right; individual results open.

Phase 6 — Robustness, Performance, Accessibility

PLAN

PLAN Phase 6 – hardening, no new features. Propose changes to: hit single-label < 5 seconds reliably (measure, then tune image size / prompt / model tier); confirm imperfect images degrade gracefully; tighten validation and error messages everywhere; and an accessibility pass (font size, contrast, tap targets, labels). List what you will measure and the target numbers. No code yet.

REVIEW

REVIEW the Phase 6 plan against the full testing checklist from the brief (valid label, mismatches, case-only, ABV/units normalization, missing/wrong-caps/correct warning, imperfect image, wrong file type, empty submit, batch summary, single-label speed). Confirm every item is covered. Finalize.

EXECUTE

EXECUTE Phase 6. Apply the tuning and validation/accessibility fixes, then run the full checklist against the DEPLOYED URL and report actual numbers (especially single-label latency). Exit check: every checklist item passes on the live URL; latency under 5 seconds is demonstrated.

Phase 7 — Deploy Verification + README + Submission

PLAN

PLAN Phase 7. List the final submission gates: public repo, live URL fully functional, no secrets in repo, README with setup/run/deployed-URL/approach/tools/assumptions/limitations. Propose a README outline and a pre-submission audit (grep the repo for hardcoded keys, confirm .env is not committed). No code yet.

REVIEW

REVIEW the Phase 7 plan. Confirm the audit would actually catch a committed key or .env, and that the README covers everything reviewers expect. Add anything missing. Finalize.

EXECUTE

EXECUTE Phase 7. Run the secret audit, write the README, and do one clean end-to-end pass on the deployed URL (single + batch + warning exact-match + an imperfect image). Exit check: audit clean, README complete, live demo works end-to-end.

Notes on Using This Playbook

- Phases 0 and 1 are deliberately first: deploying early de-risks the “works on localhost only” trap, and the comparison engine is the gradeable core, built without spending API calls.
- Government warning and secrets are the two auto-fail traps from the brief — they get dedicated coverage in Phases 1, 6, and 7.
- Keep each phase scoped. If the agent proposes work beyond the current phase during PLAN, push it back before EXECUTE.